
Algorithm control: User Manual

S. Leveque, J. R. Maddison, J. W. Pearson

CONTENTS:

1	The Software	1
1.1	Installation	1
1.2	The Control Class	1
2	Tutorials	3
2.1	Stationary Module	3
2.2	Instationary Module	12
2.3	Preconditioning Infrastructure	16
3	Test Examples from Manuscript	29
3.1	Poisson Control	29
3.2	Navier–Stokes Control	31
4	The control Module	35
4.1	Submodules	35
5	The preconditioner Module	45
5.1	Submodules	45

THE SOFTWARE

1.1 Installation

After installing Firedrake, the library can be used by adding the `control` root directory to the Python search path.

Alternatively the library can be installed via `pip`

```
python3 -m pip install .
```

run in the `control` root directory.

1.2 The Control Class

In this section, we discuss the extensions to the Firedrake system added in this software. For simplicity, we first consider the following heat control problem:

$$\min_{v,u} \frac{1}{2} \int_0^{t_f} \|v - v_d\|_{L^2(\Omega)}^2 dt + \frac{\beta}{2} \int_0^{t_f} \|u\|_{L^2(\Omega)}^2 dt$$

subject to:

$$\begin{cases} \frac{\partial v}{\partial t} - \nabla^2 v = u + f & \text{in } \Omega \times (0, t_f), \\ v = 0 & \text{in } \Omega, \\ v = 0 & \text{on } \partial\Omega \times (0, t_f), \end{cases}$$

where, for example, $\Omega = (-1, 1)^2$, $\beta = 10^{-4}$, and $t_f = 2$. The code for the definition and the solution of the previous problem is reported in Figure 1.1.

The problem is defined in a compact way by providing the weak form representing the forward differential operator in space, the boundary conditions on the state variable, the desired state, and the force function acting on the system. Note that, for stationary control problems, the callables defining the desired state and the force function accept as an input the test function of the finite element space considered, while the callable related to the forward differential operator accepts as inputs the trial function, the test function, and the current approximation of the state v . For instationary problems one has to additionally include the time t . Finally the problem is defined by instantiating an `Instationary` object.

Note that the heat control problem defined in Figure 1.1 is solved in the time interval $(0, 2)$ employing the trapezoidal rule discretization over $n_t - 1 = 9$ intervals in time, with a homogeneous initial condition for the state and regularization parameter $\beta = 10^{-4}$. The user can also provide a callable for the definition of a different initial condition, passing the argument `initial_condition`. The discretization in time can be set as the backward Euler method by passing the argument `CN=False` to the call. We would like to note that this example applies implicitly the bi-diagonal transformations employed in [2] for the derivation of the preconditioner, thus resulting in a preconditioner structure not easily available, for example, with PETSc field-split preconditioning [1].

Control problems that include incompressibility constraints are defined by passing the extra argument `space_p`, the space to which the pressure belongs. The software does not verify that inf-sup stable finite element pairs are selected.

```

from firedrake import *
from control.control import *

mesh = RectangleMesh(10, 10, 1.0, 1.0, originX=-1.0, originY=-1.0)
space_0 = FunctionSpace(mesh, "Lagrange", 1)

def forw_diff_operator(trial, test, v, t):
    return inner(grad(trial), grad(test)) * dx

def desired_state(test, t):
    space = test.function_space()
    mesh = space.mesh()
    X = SpatialCoordinate(mesh)

    v_d = Function(space, name="v_d")
    v_d.interpolate(t * cos(0.5 * pi * X[0]) * cos(0.5 * pi * X[1]))

    return inner(v_d, test) * dx, v_d

def force_f(test, t):
    space = test.function_space()
    mesh = space.mesh()
    X = SpatialCoordinate(mesh)

    f = Function(space, name="f")
    f.interpolate(cos(0.5 * pi * X[0]) * cos(0.5 * pi * X[1]))

    return inner(f, test) * dx

def bc_t(space_0, t):
    return DirichletBC(space_0, 0.0, "on_boundary")

control_instationary = Instationary(
    space_0, forw_diff_operator, desired_state=desired_state,
    force_function=force_f, bcs_v=bc_t, beta=1.0e-4, n_t=10,
    time_interval=(0.0, 2.0))

control_instationary.linear_solve()

```

Fig. 1.1: Lines of code required to solve a heat control problem.

The linear solvers consist of a Krylov subspace method preconditioned by suitable approximations of the matrices considered. The solvers allow one to employ a user-defined preconditioner, adding to the call the argument `P` and passing the solver parameters through the extra argument `solver_parameters`. The non-linear solver is based on a Picard iteration, but it can be set to a Gauss–Newton method by passing the argument `Gauss_Newton = True` to the definition of the object. We mention that the software provides the zero vector as the initial guess to the preconditioned Krylov method of choice.

2.1 Stationary Module

2.1.1 Linear Poisson Control Problem

Given $\beta > 0$, a force function f , and a desired state v_d , we consider the following linear Poisson control problem:

$$\min_{v,u} \frac{1}{2} \|v - v_d\|_{L^2(\Omega)}^2 + \frac{\beta}{2} \|u\|_{L^2(\Omega)}^2$$

subject to:

$$-\nabla^2 v = u + f \quad \text{in } \Omega := (-1, 1)^2,$$

with $v = 1$ on $\partial\Omega$. For the problem considered here, we set $v_d = \cos(\frac{\pi x_1}{2}) \cos(\frac{\pi x_2}{2}) + 1$ and $f = \frac{\pi^2}{2} \cos(\frac{\pi x_1}{2}) \cos(\frac{\pi x_2}{2})$, with analytical state solution given by $v = v_d$. Further, the adjoint variable is $\zeta = 0$. We consider a uniform grid with 10 points in each spatial direction, and employ finite elements as the spatial discretization.

In order to define the problem, we first import the `Stationary` module from the `control` class.

```
from firedrake import *
from control.control import Stationary
```

We now defined the geometry of the problem and the function space where we seek the solution.

```
mesh = RectangleMesh(10, 10, 1.0, 1.0, originX=-1.0, originY=-1.0)
space_0 = FunctionSpace(mesh, "Lagrange", 1)
```

Then, we define the desired state v_d , the force function f , and the boundary conditions to apply to v .

```
# the desired state
def desired_state(test):
    space = test.function_space()
    mesh = space.mesh()
    X = SpatialCoordinate(mesh)
    x_1 = X[0]
    x_2 = X[1]

    v_d = Function(space, name="v_d")
    v_d.interpolate(cos(0.5 * pi * x_1) * cos(0.5 * pi * x_2) + 1.0)

    return inner(v_d, test) * dx, v_d
```

```
# the force function
def force_f(test):
    space = test.function_space()
    mesh = space.mesh()
    X = SpatialCoordinate(mesh)
    x_1 = X[0]
    x_2 = X[1]

    f = Function(space, name="f")
    f.interpolate(0.5 * pi * pi * cos(0.5 * pi * x_1) * cos(0.5 * pi * x_2))

    return inner(f, test) * dx

# the boundary conditions
bcs_v = DirichletBC(space_0, 1.0, "on_boundary")
```

Finally, we define the form that represents the differential operator in space. The code will use this form to derive the adjoint operator.

```
# the forward form
def forw_diff_operator(trial, test, v):
    return inner(grad(trial), grad(test)) * dx
```

Now, we can define the Poisson control problem to be solved. This is done by passing the space from which we find the solution, the bilinear form, the desired state, the force function, the value of β , and the boundary conditions to the module `Stationary`. For this example, we set $\beta = 10^{-3}$. Note that the default option for β is 10^{-1} , so the kwarg `beta` can be avoided in the following call.

```
Poisson_control = Stationary(space_0, forw_diff_operator,
                             desired_state=desired_state,
                             force_function=force_f, beta=1.0e-3, bcs_v=bcs_v)
```

In order to solve the control problem defined above, we have to call the module `linear_solve()`. The in-built solver is preconditioned GMRES, restarted every 10 iterations. The solver runs until a relative reduction of 10^{-6} on the absolute or relative residual is achieved. In case one wishes to change the settings, one has to pass a dictionary containing the settings to the solver. For example, suppose we want to run full GMRES up to a tolerance of 10^{-8} on the absolute or relative residual, with the history of the residual printed as an output. Then, we have to pass the following dictionary to the solver.

```
solver_parameters = {"linear_solver": "gmres",
                     "maximum_iterations": 50,
                     "relative_tolerance": 1.0e-8,
                     "absolute_tolerance": 1.0e-8,
                     "monitor_convergence": True}
```

The software employs as default preconditioner a block-triangular approximation based on the so called matching strategy. The $(1, 1)$ -block is approximated with a Jacobi iteration, and each block of the Schur complement approximation is inverted inexactly with 2 cycles of an algebraic multigrid routine. The user can modify the setting by passing the kwargs `auxiliary_sp`, as follows.

For example, suppose we want to employ a fixed number of Chebyshev semi-iterations with Jacobi splitting as solver for the $(1, 1)$ -block. We first need to create a dictionary containing information about the solver for the Chebyshev semi-iteration.

```
# bounds on the eigenvalues for the preconditioned mass matrix
e_min_p = 0.5
e_max_p = 2.0

sp_11block = {"ksp_type": "chebyshev",
              "pc_type": "jacobi",
              "ksp_chebyshev_eigenvalues": f"{e_min_p:.16e}, {e_max_p:.16e}",
              "ksp_chebyshev_esteig": "0.0,0.0,0.0,0.0",
              "ksp_chebyshev_esteig_steps": 0,
              "ksp_chebyshev_esteig_noisy": False,
              "ksp_max_it": 20,
              "ksp_atol": 0.0,
              "ksp_rtol": 0.0}
```

Next, we have to create a dictionary to pass to the solver.

```
auxiliary_sp = {"sp_11block": sp_11block}
```

Finally, we can call the linear solver as follows.

```
Poisson_control.linear_solve(
    solver_parameters=solver_parameters, auxiliary_sp=auxiliary_sp,
    print_error=False, outputs=False, plots=False)
```

The state and adjoint solutions are then stored in the variables `Poisson_control._v` and `Poisson_control._zeta`, respectively.

The package allows the user to print the the L^2 discrepancy between the desired state v_d and the numerical solution by calling `print_error=True` in the code above (the default option is True). Further, `.pvd` and `.h5` outputs containing the state and the adjoint solutions; this is done by passing `outputs=True` to the call (the default option is True). Finally, plots of the solution can be obtained by setting `plots=True` (the default option is False).

We can now postprocess the solutions obtained, checking the numerical errors.

```
v = Poisson_control._v
zeta = Poisson_control._zeta

true_v = Function(space_0)
true_zeta = Function(space_0)

# evaluating the true solution
X = SpatialCoordinate(mesh)
x_1 = X[0]
x_2 = X[1]

true_v.interpolate(cos(0.5 * pi * x_1) * cos(0.5 * pi * x_2) + 1.0)
true_zeta.zero()

# evaluating the norm of the error
v_error = np.sqrt(abs(assemble(
    inner(v - true_v, v - true_v) * dx)))
zeta_error = np.sqrt(abs(assemble(
    inner(zeta - true_zeta, zeta - true_zeta) * dx)))

print(f"{v_error=}")
print(f"{zeta_error=}")
```

2.1.2 Linear Incompressible Stokes Control Problem

As an example, we consider the solution of the following stationary incompressible Stokes problem:

$$\min_{\vec{v}, \vec{u}} \frac{1}{2} \|\vec{v} - \vec{v}_d\|_{L^2(\Omega)}^2 + \frac{\beta}{2} \|\vec{u}\|_{L^2(\Omega)}^2$$

subject to:

$$\begin{cases} -\nabla^2 \vec{v} + \nabla p = \vec{u} + \vec{f} & \text{in } \Omega, \\ -\nabla \cdot \vec{v} = 0 & \text{in } \Omega, \\ \vec{v} = \vec{g} & \text{on } \partial\Omega. \end{cases}$$

We consider the lid-driven cavity problem in $\Omega = (-1, 1)^2$, with force function $\vec{f} = [0, 0]^\top$ and boundary conditions given by

$$\vec{g} = \begin{cases} [1, 0]^\top & \text{on } \partial\Omega_1 := (-1, 1) \times \{1\}, \\ [0, 0]^\top & \text{on } \partial\Omega \setminus \partial\Omega_1. \end{cases}$$

We seek $\vec{v}_d = [0, 0]^\top$ as the desired state, and set $\beta = 10^{-3}$.

As a first task, we import all the important modules. Note that the linear solver requires the user to provide as an input the nullspace of the corresponding forward stationary Stokes problem. For this reason, we have to import from the class preconditioner the module `ConstantNullspace`, as we are solving for an enclosed flow.

```
from firedrake import *
from control.preconditioner import ConstantNullspace
from control.control import Stationary
```

We can now build the mesh and define the finite element spaces from which we seek the solutions. We employ a $\mathbf{P}_2$ – P_1 finite element pair. We mention that the pressure space can be passed directly to the linear solver.

```
mesh = RectangleMesh(10, 10, 1.0, 1.0, originX=-1.0, originY=-1.0)
space_v = VectorFunctionSpace(mesh, "Lagrange", 2)
space_p = FunctionSpace(mesh, "Lagrange", 1)
```

Then, we define the boundary conditions for the velocity

```
# the boundary conditions
bcs_v = [DirichletBC(space_v, Constant((1.0, 0.0)), (4,)),
         DirichletBC(space_v, 0.0, (1, 2, 3))]
```

the forward form

```
# the forward form
def forw_diff_operator(trial, test, u):
    # spatial differential operator for the forward problem
    return inner(grad(trial), grad(test)) * dx
```

the force function, and the desired state.

Finally, we define the problem.

```
# the force function
def force_f(test):
    space = test.function_space()

    # force function
    f = Function(space)
    f.zero()

    return inner(f, test) * dx

# the desired state
def desired_state(test):
    space = test.function_space()

    # desired state
    v_d = Function(space, name="v_d")
    v_d.zero()

    return inner(v_d, test) * dx, v_d

stationary_Stokes_control = Stationary(
    space_v, forw_diff_operator, desired_state=desired_state,
    force_function=force_f, space_p=space_p, bcs_v=bcs_v)
```

In order to solve the problem, we have now to call the module `incompressible_linear_solve()`. The call requires as an input the nullspace of the corresponding forward Stokes problem we are considering here. We employ the in-built preconditioned iterative method, that is running FGMRES for 50 iterations restarted every 10 iterations, applying as the preconditioner a block-triangular matrix with the (1,1)-block approximately inverted with 5 preconditioned GMRES iterations, and a Schur complement approximation based on the block-pressure convection–diffusion preconditioner. The preconditioner for the inner GMRES solver is based on the matching strategy. The default options for the vector- and pressure-mass matrices is a Jacobi iteration, while the pressure-stiffness matrix is approximated inexactly with 1 cycle of the `hypr` multigrid routine. Note that the options can be modified by passing to the call the kwarg `auxiliary_sp`. For example, suppose we want to run the inner solver for 3 iterations stopping once the relative residual has been reduced of one order of magnitude, apply 20 Chebyshev semi-iterations to approximately invert the vector- and pressure-mass matrices, and apply 2 cycle of `hypr` as an approximation of the inverse of the pressure-stiffness matrix.

```

# option for the (1,1)-block solve
inner_solver_parameters = {
    "preconditioner": True,
    "linear_solver": "gmres",
    "maximum_iterations": 3,
    "relative_tolerance": 1.0e-1,
    "absolute_tolerance": 0.0,
    "monitor_convergence": False}

# employing Chebyshev for the (1,1)-block
e_min_v = 0.3924
e_max_v = 2.0598
sp_11block = {
    "ksp_type": "chebyshev",
    "pc_type": "jacobi",
    "ksp_chebyshev_eigenvalues": f"{e_min_v:.16e}, {e_max_v:.16e}",
    "ksp_chebyshev_esteig": "0.0,0.0,0.0,0.0",
    "ksp_chebyshev_esteig_steps": 0,
    "ksp_chebyshev_esteig_noisy": False,
    "ksp_max_it": 20,
    "ksp_atol": 0.0,
    "ksp_rtol": 0.0}

# employing Chebyshev for the pressure-mass matrix
e_min_p = 0.5
e_max_p = 2.0
sp_M_p = {
    "ksp_type": "chebyshev",
    "pc_type": "jacobi",
    "ksp_chebyshev_eigenvalues": f"{e_min_p:.16e}, {e_max_p:.16e}",
    "ksp_chebyshev_esteig": "0.0,0.0,0.0,0.0",
    "ksp_chebyshev_esteig_steps": 0,
    "ksp_chebyshev_esteig_noisy": False,
    "ksp_max_it": 20,
    "ksp_atol": 0.0,
    "ksp_rtol": 0.0}

# applying 2 cycles of the hypre multigrid
sp_K_p = {"ksp_type": "preonly",
    "pc_type": "hypre",
    "pc_hypre_type": "boomeramg",
    "ksp_max_it": 1,
    "pc_hypre_boomeramg_max_iter": 2,
    "ksp_atol": 0.0,
    "ksp_rtol": 0.0}

auxiliary_sp = {
    "sp_inner": inner_solver_parameters,
    "sp_11block": sp_11block,
    "sp_M_p": sp_M_p,
    "sp_K_p": sp_K_p}

stationary_Stokes_control.incompressible_linear_solve(
    ConstantNullspace(), auxiliary_sp=auxiliary_sp)

```

2.1.3 Non-Linear Poisson Control Problem

We consider the following stationary non-linear Poisson control problem:

$$\min_{v,u} \frac{1}{2} \|v - v_d\|_{L^2(\Omega)}^2 + \frac{\beta}{2} \|u\|_{L^2(\Omega)}^2$$

subject to:

$$-\nabla \cdot (v^2 \nabla v) = u + f \quad \text{in } \Omega := (-1, 1)^2,$$

with $v = 0$ on $\partial\Omega$, where we set $\beta = 10^{-3}$, $v_d = \cos(\frac{\pi x_1}{2}) \cos(\frac{\pi x_2}{2})$, and $f = 0$.

We consider a uniform grid with 10 points in each spatial direction, and employ P_1 finite elements as the spatial discretization.

In order to define the problem, we first import the `Stationary` module from the `control` class.

```
from firedrake import *
from control.control import Stationary
```

We now define the geometry of the problem and the function space where we seek the solution.

```
mesh = RectangleMesh(10, 10, 1.0, 1.0, originX=-1.0, originY=-1.0)
space_0 = FunctionSpace(mesh, "Lagrange", 1)
```

Then, we define the desired state v_d , the force function f , and the boundary conditions to apply to v .

```
# the desired state
def desired_state(test):
    space = test.function_space()
    mesh = space.mesh()
    X = SpatialCoordinate(mesh)
    x_1 = X[0]
    x_2 = X[1]

    v_d = Function(space, name="v_d")
    v_d.interpolate(cos(0.5 * pi * x_1) * cos(0.5 * pi * x_2))

    return inner(v_d, test) * dx, v_d

# the force function
def force_f(test):
    space = test.function_space()

    f = Function(space, name="f")
    f.zero()

    return inner(f, test) * dx

# the boundary conditions
bcs_v = DirichletBC(space_0, 0.0, "on_boundary")
```

Finally, we define the form that represents the differential operator in space. This has to be passed as the bilinear form representing the Picard linearization of the forward differential operator in space, as follows.

```
# the forward form
def forw_diff_operator(trial, test, v):
    return inner((v**2.0) * grad(trial), grad(test)) * dx
```

Now, we can define the Poisson control problem to be solved. Note that since for the problem above we have $f = 0$ we can avoid passing the `kwarg force_function` to the constructor, as by default this option assumes zero force function (similarly for the `kwarg desired_state` in the case $v_d = 0$).

```
non_linear_Poisson_control = Stationary(
    space_0, forw_diff_operator, desired_state=desired_state,
    force_function=force_f, bcs_v=bcs_v, Gauss_Newton=True)
```

With the setting above, the non-linear solver is adopting a Gauss–Newton linearization of the non-linear problem, set using `Gauss_Newton=True`. `Gauss_Newton=False` would select a Picard linearization.

Non-linear control problems are solved by calling the module `non_linear_solve()`. Default options run the solver for 10 non-linear iterations until the absolute non-linear residual is reduced by a factor of 10^{-8} or the relative non-linear residual is reduced by a factor of 10^{-5} . In order to modify the options, one has to pass specific `kwargs` to the call. In

addition, users can modify the outer solver together with the inner solvers employed to solve the linearized systems, by passing suitable options to the call.

For example, suppose we would like to run Gauss–Newton for 15 iterations, seeking for a relative reduction of 10^{-5} on both the absolute and relative non-linear residual, and we would like to employ 500 iterations of preconditioned FGMRES as the linear solver, restarted every 10 iterations, and apply the in-built preconditioner, with Chebyshev semi-iteration to approximate the inverse of the (1,1)-block. Then, we pass the following options to the module.

```
solver_parameters = {"linear_solver": "fgmres",
                    "maximum_iterations": 500,
                    "fgmres_restart": 10,
                    "relative_tolerance": 1.0e-6,
                    "absolute_tolerance": 1.0e-6,
                    "monitor_convergence": True}

# bounds on the eigenvalues for the preconditioned mass matrix
e_min_p = 0.5
e_max_p = 2.0

sp_11block = {"ksp_type": "chebyshev",
              "pc_type": "jacobi",
              "ksp_chebyshev_eigenvalues": f"{e_min_p:.16e}, {e_max_p:.16e}",
              "ksp_chebyshev_esteig": "0.0,0.0,0.0,0.0",
              "ksp_chebyshev_esteig_steps": 0,
              "ksp_chebyshev_esteig_noisy": False,
              "ksp_max_it": 20,
              "ksp_atol": 0.0,
              "ksp_rtol": 0.0}

auxiliary_sp = {"sp_11block": sp_11block}

nl_sp = {"nl_max_it": 15,
        "nl_atol": 1.0e-5,
        "nl_rtol": 1.0e-5}

non_linear_Poisson_control.non_linear_solve(
    solver_parameters=solver_parameters, auxiliary_sp=auxiliary_sp,
    nl_sp=nl_sp, outputs=False, plots=False)
```

2.1.4 Incompressible Navier–Stokes Control Problem

We now consider the case of non-linear incompressible control problems. We consider the following stationary incompressible Navier–Stokes control in $\Omega = (-1, 1)^2$ as an example:

$$\min_{\vec{v}, \vec{u}} \frac{1}{2} \|\vec{v} - \vec{v}_d\|_{L^2(\Omega)}^2 + \frac{\beta}{2} \|\vec{u}\|_{L^2(\Omega)}^2$$

subject to:

$$\begin{cases} -\nabla^2 \vec{v} + \vec{v} \cdot \nabla \vec{v} + \nabla p = \vec{u} + \vec{f} & \text{in } \Omega, \\ -\nabla \cdot \vec{v} = 0 & \text{in } \Omega, \\ \vec{v} = \vec{g} & \text{on } \partial\Omega. \end{cases}$$

We consider the lid-driven cavity problem with force function $\vec{f} = [0, 0]^\top$ and boundary conditions given by

$$\vec{g} = \begin{cases} [1, 0]^\top & \text{on } \partial\Omega_1 := (-1, 1) \times \{1\}, \\ [0, 0]^\top & \text{on } \partial\Omega \setminus \partial\Omega_1. \end{cases}$$

We seek $\vec{v}_d = [0, 0]^\top$ as the desired state, and set $\beta = 10^{-4}$.

As we have done above, we first import all the important modules, build the mesh, define the function spaces where seeking the solution together with the boundary conditions for the velocity, and define the force function and the desired state. Then, we can call the constructor of the `Stationary` class. For this problem, we apply a $\mathbf{P}_2$ – $\mathbf{P}_1$ finite element pair. As mentioned for the Stokes control problem, the pressure space can be passed directly to the non-linear solver, as we do here.

```

from firedrake import *
from control.preconditioner import ConstantNullspace
from control.control import Stationary

beta = 1.0e-4

mesh = RectangleMesh(10, 10, 1.0, 1.0, originX=-1.0, originY=-1.0)

space_v = VectorFunctionSpace(mesh, "Lagrange", 2)
space_p = FunctionSpace(mesh, "Lagrange", 1)

# the boundary conditions
bcs_v = [DirichletBC(space_v, Constant((1.0, 0.0)), (4,)),
         DirichletBC(space_v, 0.0, (1, 2, 3))]

# the forward form
def forw_diff_operator(trial, test, u):
    # viscosity
    nu = 1.0 / 100.0
    # spatial differential operator for the forward problem
    return (
        nu * inner(grad(trial), grad(test)) * dx
        + inner(dot(grad(trial), u), test) * dx)

# the desired state
def desired_state(test):
    space = test.function_space()

    # desired state
    v_d = Function(space, name="v_d")
    v_d.zero()

    return inner(v_d, test) * dx, v_d

# the force function
def force_f(test):
    space = test.function_space()

    # force function
    f = Function(space)
    f.zero()

    return inner(f, test) * dx

stationary_Navier_Stokes_control = Stationary(
    space_v, forw_diff_operator, desired_state=desired_state,
    force_function=force_f, beta=beta, bcs_v=bcs_v)

```

We employ a Picard linearization to the optimality conditions, running 10 Picard iterations and seeking a relative reduction of 10^{-5} on the relative non-linear residual. At each non-linear iteration, we employ the in-built solver, that run preconditioned FGMRES restarted every 10 iterations, and apply the matching strategy to approximately solve for the $(1, 1)$ -block and the block-pressure convection–diffusion preconditioner to approximate the Schur complement. We employ 20 Chebyshev semi-iterations to approximate the inverse of the vector- and pressure-mass matrices. In order to run the non-linear solver, we have to call the module `incompressible_non_linear_solve()`, to which the user has to provide as an input the nullspace of the corresponding forward stationary Navier–Stokes problem.

```
# employing Chebyshev for the (1,1)-block
e_min_v = 0.3924
e_max_v = 2.0598
sp_11block = {
    "ksp_type": "chebyshev",
    "pc_type": "jacobi",
    "ksp_chebyshev_eigenvalues": f"{e_min_v:.16e}, {e_max_v:.16e}",
    "ksp_chebyshev_esteig": "0.0,0.0,0.0,0.0",
    "ksp_chebyshev_esteig_steps": 0,
    "ksp_chebyshev_esteig_noisy": False,
    "ksp_max_it": 20,
    "ksp_atol": 0.0,
    "ksp_rtol": 0.0}

# employing Chebyshev for the pressure-mass matrix
e_min_p = 0.5
e_max_p = 2.0
sp_M_p = {
    "ksp_type": "chebyshev",
    "pc_type": "jacobi",
    "ksp_chebyshev_eigenvalues": f"{e_min_p:.16e}, {e_max_p:.16e}",
    "ksp_chebyshev_esteig": "0.0,0.0,0.0,0.0",
    "ksp_chebyshev_esteig_steps": 0,
    "ksp_chebyshev_esteig_noisy": False,
    "ksp_max_it": 20,
    "ksp_atol": 0.0,
    "ksp_rtol": 0.0}

auxiliary_sp = {"sp_11block": sp_11block,
               "sp_M_p": sp_M_p}

nl_sp = {"nl_max_it": 10,
        "nl_rtol": 1.0e-5}

stationary_Navier_Stokes_control.incompressible_non_linear_solve(
    ConstantNullspace(), space_p=space_p, auxiliary_sp=auxiliary_sp,
    nl_sp=nl_sp, print_error=True,
    outputs=False, plots=False)
```

2.2 Instationary Module

2.2.1 Linear Convection–Diffusion Control Problem

Given $\beta > 0$, $\Omega \subset \mathbb{R}^d$ with $d \in \{1, 2, 3\}$, $\varepsilon > 0$, and $t_f > 0$, we consider the solution of the following instationary convection–diffusion control problem:

$$\min_{v,u} \frac{1}{2} \int_0^{t_f} \|v - v_d\|_{L^2(\Omega)}^2 dt + \frac{\beta}{2} \int_0^{t_f} \|u\|_{L^2(\Omega)}^2 dt$$

subject to:

$$\frac{\partial v}{\partial t} - \varepsilon \nabla^2 v + \vec{w} \cdot \nabla v = u + f \quad \text{in } \Omega \times (0, t_f),$$

provided with suitable initial and boundary conditions.

The module `Instationary` allows the user to solve the previous problem by providing a few lines of code. For example, suppose we would like to solve the previous problem in $\Omega := (-1, 1)^2 \times (0, 2)$, starting from the initial condition

$$v_0 = \begin{cases} 1 & \text{on } \partial\Omega_1 := \{1\} \times (-1, 1), \\ 0 & \text{in } \Omega \setminus \partial\Omega_1, \end{cases}$$

and with boundary conditions given by

$$v = \begin{cases} 1 & \text{on } \partial\Omega_1 \times (0, 2), \\ 0 & \text{on } \partial\Omega_2 \times (0, 2), \end{cases}$$

where $\partial\Omega_2 := \partial\Omega \setminus \partial\Omega_1$.

For this example, we consider the wind $\vec{w}(\mathbf{x}, t) = \sin(\pi t)[2x_2(1 - x_1^2), -2x_1(1 - x_2^2)]^\top$, with diffusion parameter

$\varepsilon = \frac{1}{250}$ and regularization parameter $\beta = 10^{-2}$. Further, we set $f = 0$, and seek the desired state $v_d = e^{-10(1-x_1)}$. After importing the important modules, we first define the function space from which we seek the solution (in this example, we employ P_1 finite elements), then we define the force function, the desired state, the bilinear form, and the initial and boundary conditions. Note that, since this is a time-dependent problem, the callables accept as input the time t at which one evaluates the functions. Note also that the solver assumes that the boundary conditions on the state are applied to the same parts of the boundary on each time level.

```

from firedrake import *
from control.control import Instationary

beta = 1.0e-2

mesh = RectangleMesh(10, 10, 1.0, 1.0, originX=-1.0, originY=-1.0)

space_0 = FunctionSpace(mesh, "Lagrange", 1)

# the force function
def force_f(test, t):
    space = test.function_space()

    # force function
    f = Function(space)
    f.zero()

    return inner(f, test) * dx

# the desired state
def desired_state(test, t):
    space = test.function_space()
    mesh = space.mesh()
    X = SpatialCoordinate(mesh)

    # desired state
    v_d = Function(space, name="v_d")
    v_d.interpolate(exp(- 10.0 * (1.0 - X[0])))

    return inner(v_d, test) * dx, v_d

# the forward form
def forw_diff_operator(trial, test, u, t):
    space = test.function_space()
    mesh = space.mesh()
    X = SpatialCoordinate(mesh)
    x_1 = X[0]
    x_2 = X[1]

    # diffusion parameter
    epsilon = 1.0 / 250.0

    # wind
    w = as_vector([sin(pi * t) * 2.0 * x_2 * (1.0 - x_1 * x_1),
                  -sin(pi * t) * 2.0 * x_1 * (1.0 - x_2 * x_2)])

    # spatial differential operator for the forward problem
    return (
        epsilon * inner(grad(trial), grad(test)) * dx
        + inner(dot(grad(trial), w), test) * dx

    # auxiliary boundary conditions
    bcs_v = [DirichletBC(space_0, Constant(1.0), (4,)),
             DirichletBC(space_0, 0.0, (1, 2, 3))]

```

```
# the initial condition
def initial_condition(test):
    space = test.function_space()

    v_0 = Function(space)
    v_0.zero()

    for bc in bcs_v:
        bc.apply(v_0)

    return v_0

# the boundary conditions
def bcs_v_t(space_0, t):
    return bcs_v
```

We can now define the problem by calling the constructor `Instationary`. This is done by passing all the previous callables to it, together with the time interval for integration and the number of points n_t to employ in time (for this example, we will employ $n_t = 20$; the default option is $n_t = 10$). Note that one can also provide to the constructor the type of discretization employed by passing the kwarg `CN`. For example, suppose one would like to employ backward Euler as time discretization; then, one has to pass `CN=False` (the default option is `CN=True`). We would like to mention that, for this example, the linear system to be solved is not symmetric when applying a trapezoidal rule in time.

```
n_t = 20

instationary_convection_diffusion_control = Instationary(
    space_0, forw_diff_operator, desired_state=desired_state,
    force_function=force_f, beta=beta, CN=False, n_t=n_t,
    initial_condition=initial_condition,
    time_interval=(0.0, 2.0), bcs_v=bcs_v_t)
```

We can now call the linear solver with the module `linear_solve()`. This module accepts the same input as the `linear_solve()` module in the `Stationary` class, so one can modify the linear solver by passing the appropriate kwargs.

```
instationary_convection_diffusion_control.linear_solve(
    print_error=False, outputs=False, plots=False)
```

In case one wishes to solve a non-linear instationary problem, one has to call the module `non_linear_solve()`. The kwarg and default options are the same as for the `non_linear_solve()` of the `Stationary` class.

2.2.2 Incompressible Navier–Stokes Control Problem

Given $\beta > 0$ and $t_f > 0$, we consider the following instationary incompressible Navier–Stokes control problem:

$$\min_{\vec{v}, \vec{u}} \frac{1}{2} \int_0^{t_f} \|\vec{v} - \vec{v}_d\|_{L^2(\Omega)}^2 dt + \frac{\beta}{2} \int_0^{t_f} \|\vec{u}\|_{L^2(\Omega)}^2 dt$$

subject to:

$$\begin{cases} \frac{\partial \vec{v}}{\partial t} - \nu \nabla^2 \vec{v} + \vec{v} \cdot \nabla \vec{v} + \nabla p = \vec{u} + \vec{f} & \text{in } \Omega \times (0, t_f), \\ -\nabla \cdot \vec{v} = 0 & \text{in } \Omega \times (0, t_f), \\ \vec{v} = \vec{g} & \text{on } \partial\Omega \times (0, t_f), \\ \vec{v} = \vec{v}_0 & \text{in } \Omega. \end{cases}$$

For this example, we consider a time-dependent version of the lid-driven cavity problem in $\Omega = (-1, 1)^2$, with $\vec{v}_0 =$

$[0, 0]^\top$, $\vec{f} = [0, 0]^\top$, and

$$\vec{g} = \begin{cases} [t, 0]^\top & \text{on } \partial\Omega_1 \times (0, 1), \\ [1, 0]^\top & \text{on } \partial\Omega_1 \times (1, t_f), \\ [0, 0]^\top & \text{on } \partial\Omega_2 \times (0, t_f), \end{cases}$$

where we set $\partial\Omega_1 := (-1, 1) \times \{1\}$ and $\partial\Omega_2 := \partial\Omega \setminus \partial\Omega_1$. We seek the desired state $\vec{v}_d = [0, 0]^\top$, and integrate the problem up to $t_f = 2$ with a trapezoidal rule discretization in time. For the problem here, we set $\nu = \frac{1}{250}$ and $\beta = 10^{-4}$.

The user has to construct the problem with the module `Instationary`, then call the module `incompressible_non_linear_solve()`. The kwargs and the default options are the same as for the stationary case.

```
from firedrake import *
from control.preconditioner import ConstantNullspace
from control.control import Instationary

beta = 1.0e-4

mesh = RectangleMesh(10, 10, 1.0, 1.0, originX=-1.0, originY=-1.0)

space_v = VectorFunctionSpace(mesh, "Lagrange", 2)
space_p = FunctionSpace(mesh, "Lagrange", 1)

n_t = 5
time_interval = (0.0, 2.0)

def my_DirichletBC_t_v(space_v, t):
    if float(t) < 1.0:
        my_bcs = [DirichletBC(space_v, Constant((t, 0.0)), (4,)),
                  DirichletBC(space_v, 0.0, (1, 2, 3))]
    else:
        my_bcs = [DirichletBC(space_v, Constant((1.0, 0.0)), (4,)),
                  DirichletBC(space_v, 0.0, (1, 2, 3))]

    return my_bcs

def forw_diff_operator_v(trial, test, u, t):
    # spatial differential operator for the forward problem
    nu = 1.0 / 250.0
    return (
        nu * inner(grad(trial), grad(test)) * dx
        + inner(dot(grad(trial), u), test) * dx
    )

def desired_state_v(test, t):
    space = test.function_space()

    v_d = Function(space, name="v_d")
    v_d.zero()

    return inner(v_d, test) * dx, v_d

def initial_condition_v(test):
    space = test.function_space()

    v_0 = Function(space)
    v_0.interpolate(as_vector([0.0, 0.0]))

    return v_0

def force_f_v(test, t):
    space = test.function_space()

    # force function
    f = Function(space)
    f.interpolate(as_vector([0.0, 0.0]))

    return inner(f, test) * dx
```

```

instationary_Navier_Stokes = Instationary(
    space_v, forw_diff_operator_v, desired_state=desired_state_v,
    force_function=force_f_v, beta=beta, initial_condition=initial_condition_v,
    time_interval=time_interval, n_t=n_t,
    bcs_v=my_DirichletBC_t_v)

# employing Chebyshev for the (1,1)-block
e_min_v = 0.3924
e_max_v = 2.0598

sp_11block = {
    "ksp_type": "chebyshev",
    "pc_type": "jacobi",
    "ksp_chebyshev_eigenvalues": f"{e_min_v:.16e}, {e_max_v:.16e}",
    "ksp_chebyshev_esteig": "0.0,0.0,0.0,0.0",
    "ksp_chebyshev_esteig_steps": 0,
    "ksp_chebyshev_esteig_noisy": False,
    "ksp_max_it": 20,
    "ksp_atol": 0.0,
    "ksp_rtol": 0.0}

# employing Chebyshev for the pressure-mass matrix
e_min_p = 0.5
e_max_p = 2.0
sp_M_p = {
    "ksp_type": "chebyshev",
    "pc_type": "jacobi",
    "ksp_chebyshev_eigenvalues": f"{e_min_p:.16e}, {e_max_p:.16e}",
    "ksp_chebyshev_esteig": "0.0,0.0,0.0,0.0",
    "ksp_chebyshev_esteig_steps": 0,
    "ksp_chebyshev_esteig_noisy": False,
    "ksp_max_it": 20,
    "ksp_atol": 0.0,
    "ksp_rtol": 0.0}

auxiliary_sp = {"sp_11block": sp_11block,
               "sp_M_p": sp_M_p}

solver_parameters = {"linear_solver": "fgmres",
                    "fgmres_restart": 10,
                    "maximum_iterations": 100,
                    "relative_tolerance": 1.0e-6,
                    "absolute_tolerance": 1.0e-6,
                    "monitor_convergence": True}

nl_sp = {"nl_max_it": 10,
        "nl_atol": 1.0e-5}

instationary_Navier_Stokes.incompressible_non_linear_solve(
    ConstantNullspace(), space_p=space_p,
    solver_parameters=solver_parameters,
    auxiliary_sp=auxiliary_sp, nl_sp=nl_sp, outputs=False)

```

2.3 Preconditioning Infrastructure

2.3.1 Preconditioning Poisson Control Problems

We consider the following Poisson control problem:

$$\min_{v,u} \frac{1}{2} \|v - v_d\|_{L^2(\Omega)}^2 + \frac{\beta}{2} \|u\|_{L^2(\Omega)}^2$$

subject to:

$$\begin{cases} -\nabla^2 v = u + f & \text{in } \Omega := (-1, 1)^2, \\ v = 1 & \text{on } \partial\Omega, \end{cases}$$

with $\beta = 10^{-4}$. We set $f = \frac{\pi^2}{2} \cos(\frac{\pi x_1}{2}) \cos(\frac{\pi x_2}{2})$, and seek the desired state $v_d = \cos(\frac{\pi x_1}{2}) \cos(\frac{\pi x_2}{2}) + 1$. For this setting, the analytical state solution is given by $v = v_d$, while the adjoint variable is $\zeta = 0$.

We begin by defining the problem with the `control` class.

```

from firedrake import *
from control.control import Stationary

mesh = RectangleMesh(10, 10, 1.0, 1.0, originX=-1.0, originY=-1.0)
space_0 = FunctionSpace(mesh, "Lagrange", 1)
beta = 1.0e-4

# the desired state
def desired_state(test):
    space = test.function_space()
    mesh = space.mesh()
    X = SpatialCoordinate(mesh)
    x_1 = X[0]
    x_2 = X[1]

    v_d = Function(space, name="v_d")
    v_d.interpolate(cos(0.5 * pi * x_1) * cos(0.5 * pi * x_2) + 1.0)

    return inner(v_d, test) * dx, v_d

# the force function
def force_f(test):
    space = test.function_space()
    mesh = space.mesh()
    X = SpatialCoordinate(mesh)
    x_1 = X[0]
    x_2 = X[1]

    f = Function(space, name="f")
    f.interpolate(0.5 * pi * pi * cos(0.5 * pi * x_1) * cos(0.5 * pi * x_2))

    return inner(f, test) * dx

# the boundary conditions
bcs_v = DirichletBC(space_0, 1.0, "on_boundary")

# the forward form
def forw_diff_operator(trial, test, v):
    return inner(grad(trial), grad(test)) * dx

Poisson_control = Stationary(
    space_0, forw_diff_operator, desired_state=desired_state,
    force_function=force_f, beta=beta, bcs_v=bcs_v)

```

We now have to define the preconditioner we would like to apply. We will employ Zulehner's preconditioner for the Poisson control problem, introduced in [5].

The preconditioner is constructed by means of a callable that takes as input the `control` object, the adjoint form `D_zeta`, the forward form `D_v`, the (homogenized) boundary conditions on the state variable `bcs_v`, and the boundary conditions on the adjoint variable `bcs_zeta`. The $(1, 1)$ -block of the system to be solved is stored in the `control` module `_M_v`.

```
# convergence test for the inner linear solvers
def converged(ksp, it, rnorm):
    return it >= ksp.max_it

# Zulehner's preconditioner for the Poisson control problem
def P(self, D_zeta, D_v, bcs_v, bcs_zeta):
    sp = {"ksp_type": "preonly",
          "pc_type": "hypre",
          "pc_hypre_type": "boomeramg",
          "ksp_max_it": 1,
          "pc_hypre_boomeramg_max_iter": 2,
          "ksp_atol": 0.0,
          "ksp_rtol": 0.0}

    solver_0 = LinearSolver(
        assemble(self._M_v + (beta**0.5) * D_zeta, bcs=bcs_v),
        solver_parameters=sp)

    solver_1 = LinearSolver(
        assemble(self._M_v + (beta**0.5) * D_v, bcs=bcs_zeta),
        solver_parameters=sp)

    solver_0.ksp.addConvergenceTest(converged, prepend=True)
    solver_1.ksp.addConvergenceTest(converged, prepend=True)

    # definition of preconditioner
    def pc_linear(u_0, u_1, b_0, b_1):
        # applying Zulehner's preconditioner

        # solving for the (1,1)-block
        u_0.zero()
        solver_0.solve(u_0, b_0.copy(deepcopy=True))

        # solving for the (2,2)-block
        u_1.zero()
        solver_1.solve(u_1, b_1.copy(deepcopy=True))
        with u_1.dat.vec as u:
            u.scale(beta)

    return pc_linear
```

Since the preconditioner is symmetric positive definite, we can employ MINRES as the linear solver.

```
solver_parameters = {"linear_solver": "minres",
                     "maximum_iterations": 500,
                     "relative_tolerance": 1.0e-8,
                     "absolute_tolerance": 1.0e-8,
                     "monitor_convergence": True}
```

We can now call the `linear_solve()` module, passing the function that construct the preconditioner to the kwarg `P`, and then check the numerical error, as follows.

```

Poisson_control.linear_solve(
    P=P, solver_parameters=solver_parameters,
    outputs=False, plots=False)

v = Poisson_control._v
zeta = Poisson_control._zeta

true_v = Function(space_0)
true_zeta = Function(space_0)

# evaluating the true solution
X = SpatialCoordinate(mesh)
x_1 = X[0]
x_2 = X[1]

true_v.interpolate(cos(0.5 * pi * x_1) * cos(0.5 * pi * x_2) + 1.0)
true_zeta.zero()

# evaluating the norm of the error
v_error = np.sqrt(abs(assemble(
    inner(v - true_v, v - true_v) * dx)))
zeta_error = np.sqrt(abs(assemble(
    inner(zeta - true_zeta, zeta - true_zeta) * dx)))

print(f"{v_error=}")
print(f"{zeta_error=}")

```

In case of non-linear problems, if one wishes to employ a user-built preconditioner in the linear solver, one has to pass the callable that builds the preconditioner to the kwarg `P` in the `non_linear_solve()` module.

2.3.2 Preconditioning Stationary Incompressible Control Problems

We now consider the control of the incompressible Stokes equations. The preconditioning infrastructure works roughly as in the case of Poisson control problems, with the user needing to provide a routine for the application of the preconditioner. The routine has to take as inputs the same as for the problem in Section 2.3.1, with the addition of the nullspaces for the state and velocity variables.

To give an example, we consider the following Stokes control problem:

$$\min_{\vec{v}, \vec{u}} \frac{1}{2} \|\vec{v} - \vec{v}_d\|_{L^2(\Omega)}^2 + \frac{\beta}{2} \|\vec{u}\|_{L^2(\Omega)}^2$$

subject to:

$$\begin{cases} -\nabla^2 \vec{v} + \nabla p = \vec{u} + \vec{f} & \text{in } \Omega, \\ -\nabla \cdot \vec{v} = 0 & \text{in } \Omega, \\ \vec{v} = \vec{g} & \text{on } \partial\Omega. \end{cases}$$

We consider the lid-driven cavity problem in $\Omega = (-1, 1)^2$, with force function $\vec{f} = [0, 0]^\top$ and boundary conditions given by

$$\vec{g} = \begin{cases} [1, 0]^\top & \text{on } \partial\Omega_1 := (-1, 1) \times \{1\}, \\ [0, 0]^\top & \text{on } \partial\Omega \setminus \partial\Omega_1. \end{cases}$$

We seek $\vec{v}_d = [0, 0]^\top$ as the desired state, and set $\beta = 10^{-4}$. We first construct the problem.

```
from firedrake import *
from control.preconditioner import ConstantNullspace
from control.control import Stationary

mesh = RectangleMesh(10, 10, 1.0, 1.0, originX=-1.0, originY=-1.0)

beta = 1.0e-4

space_v = VectorFunctionSpace(mesh, "Lagrange", 2)
space_p = FunctionSpace(mesh, "Lagrange", 1)

bcs_v = [DirichletBC(space_v, Constant((1.0, 0.0)), 4),
          DirichletBC(space_v, Constant((0.0, 0.0)), (1, 2, 3))]

def forw_diff_operator(trial, test, v):
    # spatial differential operator for the forward problem
    return inner(grad(trial), grad(test)) * dx

def desired_state(test):
    space = test.function_space()

    # desired state
    v_d = Function(space)
    v_d.zero()

    return inner(v_d, test) * dx, v_d

def force_f(test):
    space = test.function_space()

    # force function
    f = Function(space)
    f.zero()

    return inner(f, test) * dx

stationary_Stokes_control = Stationary(
    space_v, forw_diff_operator, desired_state=desired_state,
    force_function=force_f, beta=beta, space_p=space_p, bcs_v=bcs_v)
```

We now build the symmetric positive definite preconditioner described in [5].

```
def converged(ksp, it, rnorm):
    return it >= ksp.max_it
```

```

def P(self, D_zeta, D_v, B, nullspace_v, nullspace_zeta, bcs_v, bcs_zeta):
    space_v = self._space_v
    space_p = self._space_p
    p_test, p_trial = TestFunction(space_p), TrialFunction(space_p)

    M_p = inner(p_trial, p_test) * dx
    K_p = inner(grad(p_trial), grad(p_test)) * dx

    # multigrid for the (1,1)-block
    sp_11block = {"ksp_type": "preonly",
                  "pc_type": "hypre",
                  "pc_hypre_type": "boomeramg",
                  "ksp_max_it": 1,
                  "pc_hypre_boomeramg_max_iter": 2,
                  "ksp_atol": 0.0,
                  "ksp_rtol": 0.0}

    # employing Chebyshev for the pressure-mass matrix
    e_min_p = 0.5
    e_max_p = 2.0
    sp_M_p = {
        "ksp_type": "chebyshev",
        "pc_type": "jacobi",
        "ksp_chebyshev_eigenvalues": f"{e_min_p:.16e}, {e_max_p:.16e}",
        "ksp_chebyshev_esteig": "0.0,0.0,0.0,0.0",
        "ksp_chebyshev_esteig_steps": 0,
        "ksp_chebyshev_esteig_noisy": False,
        "ksp_max_it": 20,
        "ksp_atol": 0.0,
        "ksp_rtol": 0.0}

    # employing multigrid for the pressure-stiffness matrix
    sp_K_p = {"ksp_type": "preonly",
              "pc_type": "hypre",
              "pc_hypre_type": "boomeramg",
              "ksp_max_it": 1,
              "pc_hypre_boomeramg_max_iter": 1,
              "ksp_atol": 0.0,
              "ksp_rtol": 0.0}

    solver_0 = LinearSolver(
        assemble(self._M_v + (beta**0.5) * D_zeta, bcs=bcs_v),
        solver_parameters=sp_11block)
    solver_0.ksp.addConvergenceTest(converged, prepend=True)

    solver_1 = LinearSolver(
        assemble(self._M_v + (beta**0.5) * D_v, bcs=bcs_zeta),
        solver_parameters=sp_11block)
    solver_1.ksp.addConvergenceTest(converged, prepend=True)

    solver_M_p = LinearSolver(
        assemble(M_p), solver_parameters=sp_M_p)
    solver_M_p.ksp.addConvergenceTest(converged, prepend=True)

    solver_K_p = LinearSolver(
        assemble(K_p), solver_parameters=sp_K_p)
    solver_K_p.ksp.addConvergenceTest(converged, prepend=True)

```

```

# definition of preconditioner
def pc_linear(u_0, u_1, b_0, b_1):
    # applying Zulehner's preconditioner

    b_00 = Cofunction(space_v.dual())
    b_01 = Cofunction(space_v.dual())

    b_00.assign(b_0.sub(0))
    b_01.assign(b_0.sub(1))

    # solving for the (1,1)-block
    u_0.sub(0).zero()
    solver_0.solve(u_0.sub(0), b_00.copy(deepcopy=True))

    # solving for the (2,2)-block
    u_0.sub(1).zero()
    solver_1.solve(u_0.sub(1), b_01.copy(deepcopy=True))
    with u_0.sub(1).dat.vec as u:
        u.scale(beta)

    del b_00
    del b_01

    b_10 = Cofunction(space_p.dual())
    b_11 = Cofunction(space_p.dual())

    # solving for the (3,3)-block
    b_10.assign(b_1.sub(0))
    u_1.sub(0).zero()
    solver_K_p.solve(u_1.sub(0), b_10.copy(deepcopy=True))

    b = Function(space_p)
    b.zero()
    solver_M_p.solve(b, b_10.copy(deepcopy=True))
    with b.dat.vec as b_v:
        b_v.scale(beta**0.5)

    with u_1.sub(0).dat.vec as b_v, \
        b.dat.vec_ro as b_1_v:
        b_v.axpy(1.0, b_1_v)
    del b

    # solving for the (4,4)-block
    b_11.assign(b_1.sub(1))
    u_1.sub(1).zero()
    solver_K_p.solve(u_1.sub(1), b_11.copy(deepcopy=True))

    b = Function(space_p)
    b.zero()
    solver_M_p.solve(b, b_11.copy(deepcopy=True))
    with b.dat.vec as b_v:
        b_v.scale(beta**0.5)

    with u_1.sub(1).dat.vec as b_v, \
        b.dat.vec_ro as b_1_v:
        b_v.axpy(1.0, b_1_v)
    del b

    with u_1.sub(1).dat.vec as b_v:
        b_v.scale(1.0 / beta)

    del b_10
    del b_11

    return pc_linear

```

Since the preconditioner is symmetric positive definite, we run MINRES as the linear solver up to a relative reduction of 10^{-8} on either the absolute or the relative residual. Then, we call the module `instationary_linear_solve()`, passing the preconditioner as `kwargs` to the call.

```

solver_parameters = {
    "linear_solver": "minres",
    "maximum_iterations": 200,
    "relative_tolerance": 1.0e-08,
    "absolute_tolerance": 1.0e-08,
    "monitor_convergence": True}

stationary_Stokes_control.incompressible_linear_solve(
    ConstantNullspace(), P=P, solver_parameters=solver_parameters,
    outputs=False, plots=False)

```

As above, in case of non-linear problems, if one wishes to employ a user-built preconditioner in the linear solver, one has to pass the callable that builds the preconditioner to the kwarg `P` in the `non_linear_solve()` module.

2.3.3 Preconditioning Instationary Control Problems

Given $\beta > 0$, $\Omega \subset \mathbb{R}^d$ with $d \in \{1, 2, 3\}$, and $t_f > 0$, we consider the solution of the following heat control problem:

$$\min_{v,u} \frac{1}{2} \int_0^{t_f} \|v - v_d\|_{L^2(\Omega)}^2 dt + \frac{\beta}{2} \int_0^{t_f} \|u\|_{L^2(\Omega)}^2 dt$$

subject to:

$$\begin{cases} \frac{\partial v}{\partial t} - \nabla^2 v = u + f & \text{in } \Omega \times (0, t_f), \\ v = g & \text{on } \partial\Omega, \\ v = v_0 & \text{in } \Omega. \end{cases}$$

We integrate the problem in $\Omega := (-1, 1)^2$ up to $t_f = 2$, and consider the analytical solution with $v = v_d := e^{t_f-t} \cos(\frac{\pi x_1}{2}) \cos(\frac{\pi x_2}{2}) + 1$ and $u = 0$, and derive the force function f , the boundary conditions g , and the initial condition v_0 accordingly. For this example, we set $\beta = 10^{-4}$.

```

from firedrake import *
from control.control import Instationary

mesh = RectangleMesh(10, 10, 1.0, 1.0, originX=-1.0, originY=-1.0)

beta = 1.0e-4
t_f = 2.0
n_t = 5

space_0 = FunctionSpace(mesh, "Lagrange", 1)

def forw_diff_operator(trial, test, v, t):
    # spatial differential operator for the forward problem
    return inner(grad(trial), grad(test)) * dx

def desired_state(test, t):
    space = test.function_space()
    mesh = space.mesh()
    X = SpatialCoordinate(mesh)

    # desired state
    v_d = Function(space)
    v_d.interpolate(exp(t_f - t) * cos(0.5 * pi * X[0]) * cos(0.5 * pi * X[1]) + 1.0)

    return inner(v_d, test) * dx, v_d

```

```
def force_f(test, t):
    space = test.function_space()
    mesh = space.mesh()
    X = SpatialCoordinate(mesh)

    c = Constant(exp(t_f - t) * (-1.0 + 0.5 * pi * pi))

    # force function
    f = Function(space)
    f.interpolate(c * cos(0.5 * pi * X[0]) * cos(0.5 * pi * X[1]))

    return inner(f, test) * dx

def initial_condition(test):
    space = test.function_space()
    mesh = space.mesh()
    X = SpatialCoordinate(mesh)

    # desired state
    v_0 = Function(space)
    v_0.interpolate(exp(t_f) * cos(0.5 * pi * X[0]) * cos(0.5 * pi * X[1]) + 1.0)

    return v_0

def bcs_v_t(space_0, t):
    bcs_v_t = DirichletBC(space_0, 1.0, "on_boundary")
    return bcs_v_t

heat_control = Instationary(
    space_0, forw_diff_operator, desired_state=desired_state,
    force_function=force_f, beta=beta, n_t=n_t,
    initial_condition=initial_condition,
    time_interval=(0.0, t_f), bcs_v=bcs_v_t)
```

We now construct the preconditioner. This has to take as input the `control` object, the (1, 2)- and the (2, 1)-blocks, the full space-time space for the solutions v and ζ , and the corresponding boundary conditions. The (1, 1)- and the (2, 2)-blocks can be constructed from the form `self._M_v`, while the time step can be recovered easily. As we mentioned, since this example is just included for the purpose of explaining how to build a preconditioner, we will be employing the identity matrix as the preconditioner, meaning that in practice no preconditioner is applied. This is not recommended, as the preconditioned Krylov method will probably not converge to the given tolerance.

```
def P(self, block_01, block_10,
    full_space_v_help, bcs_v, bcs_zeta):
    # definition of preconditioner
    def pc_linear(u_0, u_1, b_0, b_1):
        u_0.assign(b_0.riesz_representation("l2"))
        u_1.assign(b_1.riesz_representation("l2"))

    return pc_linear
```

We can now solve the problem by passing the preconditioner as `kwarg` to the module `linear_solve`.

```
heat_control.linear_solve(P=P, outputs=False, plots=False)
```

In case of non-linear control problems, the preconditioner can be given as input to the module `non_linear_solve()`, by passing it to the `kwarg` `P`.

In a similar way, for incompressible control problems, either in the linear or non-linear setting, one has to give as input to the corresponding solver a function for the construction of the preconditioner to the `kwarg` `P`. In this case, the function should have the following form.

```

def P(self, block_00_int, block_01_int,
    block_10_int, block_11_int, B,
    full_nullspace_v, full_nullspace_zeta,
    bcs_v, bcs_zeta):
    # definition of preconditioner
    def pc_linear(u_0, u_1, b_0, b_1):
        u_0.assign(b_0.riesz_representation("l2"))
        u_1.assign(b_1.riesz_representation("l2"))

    return pc_linear

```

In the previous function, the input `block_00_int`, `block_01_int`, `block_10_int`, and `block_11_int` contain the dictionary of the inner blocks coupling the state and adjoint velocities; further, `full_nullspace_v` and `full_nullspace_zeta` contain the nullspaces for the two variables; finally, `bcs_v` and `bcs_zeta` contain the homogenized boundary conditions on v and ζ . This has to be built in this way in case the user would like to apply an inner solver with the class `MultiBlockSystem` for the system of the coupled velocities.

2.3.4 Preconditioner Module

This module constructs a generalized saddle-point system and then solves it by applying a suitable solver. Suppose we want to solve the following incompressible Stokes equations:

$$\begin{cases} -\nabla^2 \vec{v} + \nabla p = \vec{f} & \text{in } \Omega, \\ \nabla \cdot \vec{v} = 0 & \text{in } \Omega, \\ \vec{v} = \vec{g} & \text{on } \partial\Omega, \end{cases}$$

where, for example, $\Omega = (-1, 1)^2$, $\vec{f} = \vec{0}$, and

$$\vec{g} = \begin{cases} [1, 0]^\top & \text{on } \partial\Omega_1 := (-1, 1) \times \{1\}, \\ [0, 0]^\top & \text{on } \partial\Omega \setminus \partial\Omega_1. \end{cases}$$

Suppose we want to solve the problem on a uniform mesh and employ a $\mathbf{P}_2\text{--}P_1$ finite element pair. After including the important modules, we begin by defining the mesh and these function spaces, as follows.

```

from firedrake import *
from control.preconditioner import *

mesh = RectangleMesh(10, 10, 1.0, 1.0, originX=-1.0, originY=-1.0)

space_v = VectorFunctionSpace(mesh, "Lagrange", 2)
space_p = FunctionSpace(mesh, "Lagrange", 1)

```

The system is defined by passing the spaces and dictionaries that contain the forms representing the blocks of the system, together with the nullspaces of each variable. Specifically, for the test problem above, we would write

```
v_test, v_trial = TestFunction(space_v), TrialFunction(space_v)
p_test, p_trial = TestFunction(space_p), TrialFunction(space_p)

block_00 = {}
block_00[(0, 0)] = inner(grad(v_test), grad(v_trial)) * dx
block_01 = {}
block_01[(0, 0)] = - inner(p_trial, div(v_test)) * dx
block_10 = {}
block_10[(0, 0)] = - inner(div(v_trial), p_test) * dx
block_11 = {}
block_11[(0, 0)] = None

bcs_v = [DirichletBC(space_v, Constant((1.0, 0.0)), (4,)),
          DirichletBC(space_v, 0.0, (1, 2, 3))]

nullspace_v = DirichletBCNullspace(bcs_v)
nullspace_p = ConstantNullspace()
```

Then, the system is constructed via:

```
system = MultiBlockSystem(
    space_v, space_p,
    block_00=block_00, block_01=block_01,
    block_10=block_10, block_11=block_11,
    n_blocks_00=1, n_blocks_11=1,
    nullspace_0=nullspace_v, nullspace_1=nullspace_p)
```

Default options for `n_blocks_00` and `n_blocks_11` are both 1. These represent the number of sub-blocks in the (1,1)- and (2,2)-blocks of the generalized saddle-point system.

After this, we construct the right-hand side, as follows.

```
v_inhom = Function(space_v)
apply_bcs(bcs_v, v_inhom)

f = Function(space_v)
f = assemble(- action(block_00[(0, 0)], v_inhom))
apply_bcs(homogenize(bcs_v), f)

div_v = Function(space_p)
div_v = assemble(- action(block_10[(0, 0)], v_inhom))

b_0 = Cofunction(space_v.dual(), name="b_0")
b_1 = Cofunction(space_p.dual(), name="b_1")

b_0.assign(f)
b_1.assign(div_v)
```

Finally, we can solve the linear system. We run MINRES up to a relative reduction of 10^{-6} on the relative and absolute residual, employing the preconditioner described in [4].

```

solver_parameters = {"linear_solver": "minres",
                     "maximum_iterations": 100,
                     "relative_tolerance": 1.0e-6,
                     "absolute_tolerance": 1.0e-6,
                     "monitor_convergence": True}

# Used to avoid convergence errors when max_it is reached
def converged(ksp, it, rnorm):
    return it >= ksp.max_it

sp_11block = {"ksp_type": "preonly",
              "pc_type": "hypre",
              "pc_hypre_type": "boomeramg",
              "ksp_max_it": 1,
              "pc_hypre_boomeramg_max_iter": 2,
              "ksp_atol": 0.0,
              "ksp_rtol": 0.0}

sp_Schur = {"ksp_type": "preonly",
            "pc_type": "jacobi",
            "ksp_max_it": 1,
            "ksp_atol": 0.0,
            "ksp_rtol": 0.0}

K_v = inner(grad(v_test), grad(v_trial)) * dx
M_p = inner(p_test, p_trial) * dx

solver_0 = LinearSolver(
    assemble(K_v, bcs=homogenize(bcs_v)),
    solver_parameters=sp_11block)
solver_1 = LinearSolver(
    assemble(M_p),
    solver_parameters=sp_Schur)
solver_0.ksp.addConvergenceTest(converged, prepend=True)
solver_1.ksp.addConvergenceTest(converged, prepend=True)

def pc_fn(u_0, u_1, b_0, b_1):
    # solving for the (1,1)-block
    u_0.zero()
    solver_0.solve(u_0, b_0.copy(deepcopy=True))

    # solving for the Schur complement approximation
    u_1.zero()
    solver_1.solve(u_1, b_1.copy(deepcopy=True))

v_sol = Function(space_v)
p_sol = Function(space_p)

# solving linear system
system.solve(
    u_v_sol, u_p_sol, b_0, b_1,
    solver_parameters=solver_parameters,
    pc_fn=pc_fn)

```


TEST EXAMPLES FROM MANUSCRIPT

In this chapter, we present the code required to solve the two problems defined in the numerical examples of the manuscript, specifically the control of the Poisson equation and the control of the time-dependent Navier–Stokes equation with a trapezoidal rule discretization in time.

The code is based on the work "Automatic Differentiation for All-at-once Systems Arising in Certain PDE-Constrained Optimization Problems" by Santolo Leveque, James R. Maddison, and John W. Pearson. The code in Fig. 1.1 can be found in

- ‘paper_examples/heat_control.py’

while the tests in Section 4.1 and Section 4.2 of the paper can be obtained by running

- ‘paper_examples/2D_Poisson_control.py’ (Table 2 of the work)
- ‘paper_examples/3D_Poisson_control.py’ (Table 3 of the work)
- ‘paper_examples/NS_control_1.py’ (Table 4 of the work)
- ‘paper_examples/NS_control_2.py’ (Table 4 of the work)
- ‘paper_examples/NS_control_3.py’ (Table 4 of the work)

3.1 Poisson Control

Given the domain $\Omega \subset \mathbb{R}^d$, for $d = 2, 3$, and $\beta > 0$, we solve the following linear Poisson control problem:

$$\min_{v \in H^1(\Omega), u \in L^2(\Omega)} \frac{1}{2} \|v - v_d\|_{L^2(\Omega)}^2 + \frac{\beta}{2} \|u\|_{L^2(\Omega)}^2$$

subject to:

$$\begin{cases} -\nabla^2 v = u & \text{in } \Omega, \\ v = 1 & \text{on } \partial\Omega. \end{cases}$$

We seek as a desired state

$$v_d = \prod_{i=1}^d \cos\left(\frac{\pi x_i}{2}\right) + 1,$$

and we set $\Omega = (-1, 1)^d$. The code for the solution of the above Poisson control problem for $d = 2$ and $\beta = 10^{-4}$ is given in Figure 3.1, while we report the code for the solution of the 3D case with the same value of β in Figure 3.2.

```

from firedrake import *
from control.control import *

mesh = RectangleMesh(10, 10, 2.0, 2.0)
space_0 = FunctionSpace(mesh, "Lagrange", 1)

def forw_diff_operator(trial, test, v):
    return inner(grad(trial), grad(test)) * dx

def desired_state(test):
    space = test.function_space()
    mesh = space.mesh()
    X = SpatialCoordinate(mesh)
    x = X[0] - 1.0
    y = X[1] - 1.0

    v_d = Function(space, name="v_d")
    v_d.interpolate(cos(0.5 * pi * x) * cos(0.5 * pi * y) + 1.0)

    return inner(v_d, test) * dx, v_d

bc = DirichletBC(space_0, 1.0, "on_boundary")

control_stationary = Stationary(
    space_0, forw_diff_operator, desired_state=desired_state,
    beta=1.0e-4, bcs_v=bc)

control_stationary.linear_solve()

```

Fig. 3.1: Lines of code required to solve a 2D Poisson control problem.

```

from firedrake import *
from control.control import *

mesh = BoxMesh(10, 10, 10, 2.0, 2.0, 2.0)
space_0 = FunctionSpace(mesh, "Lagrange", 1)

def forw_diff_operator(trial, test, v):
    return inner(grad(trial), grad(test)) * dx

def desired_state(test):
    space = test.function_space()
    mesh = space.mesh()
    X = SpatialCoordinate(mesh)
    x = X[0] - 1.0
    y = X[1] - 1.0
    z = X[2] - 1.0

    v_d = Function(space, name="v_d")
    v_d.interpolate(
        cos(0.5 * pi * x) * cos(0.5 * pi * y) * cos(0.5 * pi * z) + 1.0)

    return inner(v_d, test) * dx, v_d

bc = DirichletBC(space_0, 1.0, "on_boundary")

control_stationary = Stationary(
    space_0, forw_diff_operator, desired_state=desired_state,
    beta=1.0e-4, bcs_v=bc)

control_stationary.linear_solve()

```

Fig. 3.2: Lines of code required to solve a 3D Poisson control problem.

3.2 Navier–Stokes Control

We solve the time-dependent Navier–Stokes control problem defined in [3, Section 6.3], which is given by the following instationary lid-driven cavity flow problem:

$$\min_{\vec{v}, \vec{u}} \frac{1}{2} \int_0^{t_f} \|\vec{v} - \vec{v}_d\|_{L^2(\Omega)}^2 dt + \frac{\beta}{2} \int_0^{t_f} \|\vec{u}\|_{L^2(\Omega)}^2 dt$$

subject to:

$$\begin{cases} \frac{\partial \vec{v}}{\partial t} - \nu \nabla^2 \vec{v} + \vec{v} \cdot \nabla \vec{v} = \vec{u} & \text{in } \Omega \times (0, t_f), \\ \vec{v} = \vec{0} & \text{in } \Omega, \\ \vec{v} = \vec{g} & \text{on } \partial\Omega \times (0, t_f), \end{cases}$$

where $\Omega = (-1, 1)^2$. For this test, we set $\beta = 10^{-3}$, $t_f = 2$, and consider the following boundary conditions:

$$\vec{g} = \begin{cases} [t, 0]^\top & \text{on } \partial\Omega_1 \times (0, 1), \\ [1, 0]^\top & \text{on } \partial\Omega_1 \times [1, 2), \\ [0, 0]^\top & \text{on } (\partial\Omega \setminus \partial\Omega_1) \times (0, 2), \end{cases}$$

with $\partial\Omega_1 := (-1, 1) \times \{1\}$. Further, for $\mathbf{x} = (x_1, x_2)$, we seek as the desired state

$$\vec{v}_d = \begin{cases} c_1 \cos(\frac{\pi t}{2}) [(\frac{100}{99})^2 x_2, -(\frac{100}{49})^2 (x_1 - \frac{1}{2})]^\top & \text{if } c_1 \geq 0, \\ c_2 \cos(\frac{\pi t}{2}) [-(\frac{100}{99})^2 x_2, (\frac{100}{49})^2 (x_1 + \frac{1}{2})]^\top & \text{if } c_2 \geq 0, \\ [0, 0]^\top & \text{otherwise,} \end{cases}$$

where we set

$$\begin{aligned} c_1 &= 1 - \sqrt{(\frac{100}{49}(x_1 - \frac{1}{2}))^2 + (\frac{100}{99}x_2)^2}, \\ c_2 &= 1 - \sqrt{(\frac{100}{49}(x_1 + \frac{1}{2}))^2 + (\frac{100}{99}x_2)^2}. \end{aligned}$$

We time discretization we employ is a trapezoidal rule. The code for the solution of this problem is given in Figures 3.3–3.4.

```
from firedrake import *
from control.control import *
from control.preconditioner import ConstantNullspace
import ufl

mesh = RectangleMesh(10, 10, 2.0, 2.0)
space_v = VectorFunctionSpace(mesh, "Lagrange", 2)
space_p = FunctionSpace(mesh, "Lagrange", 1)

def my_DirichletBC_t_v(space_v, t):
    if float(t) < 1.0:
        my_bcs = [DirichletBC(space_v, Constant((t, 0.0)), (4,)),
                  DirichletBC(space_v, 0.0, (1, 2, 3))]
    else:
        my_bcs = [DirichletBC(space_v, Constant((1.0, 0.0)), (4,)),
                  DirichletBC(space_v, 0.0, (1, 2, 3))]
    return my_bcs

def forw_diff_operator_v(trial, test, u, t):
    nu = 1.0 / 50.0
    return (nu * inner(grad(trial), grad(test)) * dx
            + inner(dot(u, grad(trial)), test) * dx)

def desired_state_v(test, t):
    space_v = test.function_space()
    mesh = space_v.mesh()
    X = SpatialCoordinate(mesh)
    x = X[0] - 1.0
    y = X[1] - 1.0

    a = (100.0 / 49.0) ** 2
    b = (100.0 / 99.0) ** 2

    c_1 = 1.0 - sqrt(a * ((x - 0.5) ** 2) + b * (y ** 2))
    c_2 = 1.0 - sqrt(a * ((x + 0.5) ** 2) + b * (y ** 2))
    v_d = Function(space_v, name="v_d")
    v_d.interpolate(
        ufl.conditional(c_1 >= 0.0,
                        c_1 * cos(pi * t / 2.0) * as_vector((b * y,
                                                              -a * (x - 0.5))),
                        ufl.conditional(c_2 >= 0.0,
                        c_2 * cos(pi * t / 2.0) * as_vector((-b * y,
                                                              a * (x + 0.5))),
                        as_vector((0.0, 0.0))),
    )
    return inner(v_d, test) * dx, v_d

control_instationary = Instationary(
    space_v, forw_diff_operator_v, desired_state=desired_state_v,
    time_interval=(0.0, 2.0), n_t=10, bcs_v=my_DirichletBC_t_v)
```

Fig. 3.3: Lines of code required to solve a time-dependent incompressible Navier–Stokes control problem, with the trapezoidal rule in time (code snippet 1/2).

```

e_min_v = 0.3924
e_max_v = 2.0598
sp_11block = {
    "ksp_type": "chebyshev",
    "pc_type": "jacobi",
    "ksp_chebyshev_eigenvalues": f"{e_min_v:.16e}, {e_max_v:.16e}",
    "ksp_chebyshev_esteig": "0.0,0.0,0.0,0.0",
    "ksp_chebyshev_esteig_steps": 0,
    "ksp_chebyshev_esteig_noisy": False,
    "ksp_max_it": 20,
    "ksp_atol": 0.0,
    "ksp_rtol": 0.0}

e_min_p = 0.5
e_max_p = 2.0
sp_M_p = {
    "ksp_type": "chebyshev",
    "pc_type": "jacobi",
    "ksp_chebyshev_eigenvalues": f"{e_min_p:.16e}, {e_max_p:.16e}",
    "ksp_chebyshev_esteig": "0.0,0.0,0.0,0.0",
    "ksp_chebyshev_esteig_steps": 0,
    "ksp_chebyshev_esteig_noisy": False,
    "ksp_max_it": 20,
    "ksp_atol": 0.0,
    "ksp_rtol": 0.0}

auxiliary_sp = {
    "sp_11block": sp_11block,
    "sp_M_p": sp_M_p}

control_instationary.incompressible_non_linear_solve(
    ConstantNullspace(), space_p=space_p,
    auxiliary_sp=auxiliary_sp)

```

Fig. 3.4: Lines of code required to solve a time-dependent incompressible Navier–Stokes control problem, with the trapezoidal rule in time (code snippet 2/2).

THE CONTROL MODULE

4.1 Submodules

4.1.1 The `control.Stationary` Class

This class solves a stationary control problem.

Submodules

```
control.Stationary(space_v, forward_form, desired_state=None, force_function=None, *,
beta=1.0e-3, space_p=None, Gauss_Newton=False, bcs_v=None)
```

Create `Stationary`.

Input:

- `space_v` – space to which the solution belongs
- `forward_form` – form that represents the differential operator in space
- `desired_state` – desired state, defaults to zero
- `force_function` – force function acting on the system, defaults to zero
- `beta` – regularization parameter
- `space_p` – pressure space (only for incompressible problems)
- `Gauss_Newton` – if `True`, a Gauss–Newton linearization is employed, otherwise a Picard linearization is applied
- `bcs_v` – boundary conditions on the state

```
control.Stationary.set_space_p(space_p)
```

Set `space_p` as the pressure space.

Input:

- `space_p` – the new pressure space

```
control.Stationary.set_v(v_new)
```

Set `v_new` as the state variable.

Input:

- `v_new` – the new approximation of the state solution

`control.Stationary.set_zeta(zeta_new)`

Set `zeta_new` as the adjoint variable.

Input:

- `zeta_new` – the new approximation of the adjoint solution

`control.Stationary.set_p(p_new)`

Set `p_new` as the (state) pressure variable.

Input:

- `p_new` – the new approximation of the (state) pressure solution

`control.Stationary.set_mu(mu_new)`

Set `mu_new` as the adjoint pressure variable.

Input:

- `mu_new` – the new approximation of the adjoint pressure solution

`control.Stationary.print_error()`

Print the L^2 norm of the difference between the numerical solution and the desired state.

`control.Stationary.construct_D_v(v_trial, v_test, v_old, *, non_linear_res=False)`

Construct the discretized forward form.

Input:

- `v_trial` – trial function
- `v_test` – test function
- `v_old` – approximation of the state solution
- `non_linear_res` – if `True`, the form is employed in the evaluation of the non-linear residual

Output:

- `D_v` – the discretized forward form

`control.Stationary.construct_f(v_test, D_v, bcs_v, *, v_inhom=None)`

Construct the vector containing the force function.

Input:

- `v_test` – test function
- `D_v` – discretized forward form
- `bcs_v` – homogenization of the boundary conditions on the state variable
- `v_inhom` – function that is zero in the interior of the domain and interpolates the state variable on the boundary

Output:

- `f` – the discretized force function

```
control.Stationary.construct_v_d(bcs_v, *, v_inhom=None)
```

Construct the vector containing the desired state.

Input:

- `bcs_v` – homogenization of the boundary conditions on the state variable
- `v_inhom` – function that is zero in the interior of the domain and interpolates the state variable on the boundary

Output:

- `v_d` – the discretized desired state

```
control.Stationary.construct_pc(auxiliary_sp, bcs_v, bcs_zeta, D_v, D_zeta)
```

Construct the preconditioner, based on the matching strategy.

Input:

- `auxiliary_sp` – auxiliary solver parameters for inner blocks
- `bcs_v` – homogenized boundary conditions for the state variable
- `bcs_zeta` – homogenized boundary conditions for the adjoint variable
- `D_v` – discretized forward form
- `D_zeta` – discretized adjoint form

Output:

- `pc_linear` – preconditioner to employ within the Krylov method

```
control.Stationary.non_linear_res_eval(v_d, f, v_old, zeta_old, D_v, D_zeta, M_zeta,
bcs_v, bcs_zeta)
```

Construct the non-linear residual.

Input:

- `v_d` – desired state
- `f` – force function
- `v_old` – approximation of the state variable
- `zeta_old` – approximation of the adjoint variable
- `D_v` – discretized forward form
- `D_zeta` – discretized adjoint form
- `M_zeta` – (2,2)-block
- `bcs_v` – homogenized boundary conditions for the state variable
- `bcs_zeta` – homogenized boundary conditions for the adjoint variable

Output:

- `rhs_0` – non-linear residual (adjoint equation)
- `rhs_1` – non-linear residual (state equation)

```
control.Stationary.linear_solve(*, P=None, solver_parameters=None, auxiliary_sp={},  
v_d=None, f=None, print_error=True, outputs=True, plots=False)
```

Module for the solution of linear control problems.

Input:

- `P` – preconditioner to apply within the Krylov method (if `None`, default option is employed)
- `solver_parameters` – parameters to pass to the Krylov solver
- `auxiliary_sp` – auxiliary parameters for setting solvers of inner blocks
- `v_d` – when solving non-linear problems, `v_d` is the non-linear residual (adjoint equation)
- `f` – when solving non-linear problems, `f` is the non-linear residual (state equation)
- `print_error` – if `True`, the L^2 discrepancy between the desired state and the numerical solution is printed
- `outputs` – if `True`, output is generated
- `plots` – if `True`, plots of the solutions are generated

```
control.Stationary.non_linear_solve(*, P=None, solver_parameters=None, auxiliary_sp={},  
nl_sp=None, print_error=True, outputs=True, plots=False)
```

Module for the solution of non-linear control problems.

Input:

- `P` – preconditioner to apply within the Krylov method (if `None`, default option is employed)
- `solver_parameters` – parameters to pass to the Krylov solver
- `auxiliary_sp` – auxiliary parameters for setting solvers of inner blocks
- `nl_sp` – parameters for the non-linear solver
- `print_error` – if `True`, the L^2 discrepancy between the desired state and the numerical solution is printed
- `outputs` – if `True`, output is generated
- `plots` – if `True`, plots of the solutions are generated

```
control.Stationary.incompressible_linear_solve(nullspace_p, *, space_p=None, P=None,  
solver_parameters=None, auxiliary_sp={}, v_d=None, f=None, div_v=None, div_zeta=None,  
print_error=True, outputs=True, plots=False)
```

Module for the solution of linear incompressible control problems.

Input:

- `nullspace_p` – nullspace of the corresponding forward stationary incompressible problem
- `space_p` – pressure space, if not passed to the constructor
- `P` – preconditioner to apply within the Krylov method (if `None`, default option is employed)
- `solver_parameters` – parameters to pass to the Krylov solver
- `auxiliary_sp` – auxiliary parameters for setting solvers of inner blocks
- `v_d` – when solving non-linear problems, `v_d` is the non-linear residual (adjoint equation)
- `f` – when solving non-linear problems, `f` is the non-linear residual (state equation)

- `div_v` – when solving non-linear problems, `div_v` is the non-linear residual (incompressibility constraint on the state variable)
- `div_zeta` – when solving non-linear problems, `div_zeta` is the non-linear residual (incompressibility constraint on the adjoint variable)
- `print_error` – if `True`, the L^2 discrepancy between the desired state and the numerical solution is printed
- `outputs` – if `True`, output is generated
- `plots` – if `True`, plots of the solutions are generated

```
control.Stationary.incompressible_non_linear_solve(nullspace_p, *, space_p=None, P=None,
solver_parameters=None, auxiliary_sp={}, nl_sp=None, print_error=True, outputs=True,
plots=False)
```

Module for the solution of non-linear incompressible control problems.

Input:

- `nullspace_p` – nullspace of the corresponding forward stationary incompressible problem
- `space_p` – pressure space, if not passed to the constructor
- `P` – preconditioner to apply within the Krylov method (if `None`, default option is employed)
- `solver_parameters` – parameters to pass to the Krylov solver
- `auxiliary_sp` – auxiliary parameters for setting solvers of inner blocks
- `nl_sp` – parameters for the non-linear solver
- `print_error` – if `True`, the L^2 discrepancy between the desired state and the numerical solution is printed
- `outputs` – if `True`, output is generated
- `plots` – if `True`, plots of the solutions are generated

4.1.2 The control.Instationary Class

This class solves an instationary control problem.

Submodules

```
control.Instationary(space_v, forward_form, desired_state=None, force_function=None, *,
beta=1.0e-3, space_p=None, Gauss_Newton=False, CN=True, n_t=20, initial_condition=None,
time_interval=(0.0, 1.0), bcs_v=None)
```

Create `Instationary`.

Input:

- `space_v` – space to which the solution belongs
- `forward_form` – form that represents the differential operator in space
- `desired_state` – desired state, defaults to zero
- `force_function` – force function acting on the system, defaults to zero
- `beta` – regularization parameter
- `space_p` – pressure space (only for incompressible problems)

- `Gauss_Newton` – if `True`, a Gauss–Newton linearization is employed, otherwise a Picard linearization is applied
- `CN` – if `True`, the trapezoidal rule is employed as discretization in time
- `n_t` – number of points in time
- `initial_condition` – initial condition
- `time_interval` – interval of time integration
- `bcs_v` – boundary conditions on the state

`control.Instationary.set_space_p(space_p)`

Set `space_p` as the pressure space.

Input:

- `space_p` – the new pressure space

`control.Instationary.set_v(v_new)`

Set `v_new` as the state variable.

Input:

- `v_new` – the new approximation of the state solution

`control.Instationary.set_zeta(zeta_new)`

Set `zeta_new` as the adjoint variable.

Input:

- `zeta_new` – the new approximation of the adjoint solution

`control.Instationary.set_p(p_new)`

Set `p_new` as the (state) pressure variable.

Input:

- `p_new` – the new approximation of the (state) pressure solution

`control.Instationary.set_mu(mu_new)`

Set `mu_new` as the adjoint pressure variable.

Input:

- `mu_new` – the new approximation of the adjoint pressure solution

`control.Instationary.print_error(tau)`

Print the L^2 norm of the difference between the numerical solution and the desired state.

`control.Instationary.construct_D_v(v_trial, v_test, v_n_help, t, *, non_linear_res=False)`

Construct the discretized forward form.

Input:

- `v_trial` – trial function
- `v_test` – test function
- `v_n_help` – approximation of the state solution at time `t`
- `t` – time point at which we are evaluating the form
- `non_linear_res` – if True, the form is employed in the evaluation of the non-linear residual

Output:

- `D_v` – discretized forward form

`control.Instationary.construct_f(v_test)`

Construct the vector containing the force function.

Input:

- `v_test` – test function

Output:

- `f` – the discretized force function

`control.Instationary.construct_v_d()`

Construct the vector containing the desired state.

Output:

- `v_d` – the discretized desired state

`control.Instationary.construct_pc(auxiliary_sp, full_space_v, bcs_v, bcs_zeta, block_01, block_10, epsilon=None)`

Construct the preconditioner, based on the matching strategy.

Input:

- `auxiliary_sp` – auxiliary solver parameters for inner blocks
- `full_space_v` – full space for time integration
- `bcs_v` – homogenized boundary conditions for the state variable
- `bcs_zeta` – homogenized boundary conditions for the adjoint variable
- `block_01` – (1,2)-block of the linear system, containing discretized adjoint forms
- `block_10` – (2,1)-block of the linear system, containing discretized state forms
- `epsilon` – parameters employed for the construction of the preconditioner for the backward Euler discretization

Output:

- `pc_linear` – preconditioner to employ within the Krylov method

`control.Instationary.non_linear_res_eval(full_space_v, v_old, zeta_old, v_0, v_d, f, M_v, bcs_v, bcs_zeta)`

Construct the non-linear residual.

Input:

- `full_space_v` – full space for time integration
- `v_old` – approximation of the state variable
- `zeta_old` – approximation of the adjoint variable
- `v_0` – initial condition on the state variable
- `v_d` – desired state
- `f` – force function
- `M_v` – mass matrix on the state space
- `bcs_v` – homogenized boundary conditions for the state variable
- `bcs_zeta` – homogenized boundary conditions for the adjoint variable

Output:

- `rhs_0` – non-linear residual (adjoint equation)
- `rhs_1` – non-linear residual (state equation)

```
control.Instationary.linear_solve(*, P=None, solver_parameters=None, auxiliary_sp={},  
v_d=None, f=None, print_error=True, outputs=True, plots=False)
```

Module for the solution of linear control problems.

Input:

- `P` – preconditioner to apply within the Krylov method (if `None`, default option is employed)
- `solver_parameters` – parameters to pass to the Krylov solver
- `auxiliary_sp` – auxiliary parameters for setting solvers of inner blocks
- `v_d` – when solving non-linear problems, `v_d` is the non-linear residual (adjoint equation)
- `f` – when solving non-linear problems, `f` is the non-linear residual (state equation)
- `print_error` – if `True`, the L^2 discrepancy between the desired state and the numerical solution is printed
- `outputs` – if `True`, output is generated
- `plots` – if `True`, plots of the solutions are generated

```
control.Instationary.non_linear_solve(*, P=None, solver_parameters=None, auxiliary_sp={},  
nl_sp=None, print_error=True, outputs=True, plots=False)
```

Module for the solution of non-linear control problems.

Input:

- `P` – preconditioner to apply within the Krylov method (if `None`, default option is employed)
- `solver_parameters` – parameters to pass to the Krylov solver
- `auxiliary_sp` – auxiliary parameters for setting solvers of inner blocks
- `nl_sp` – parameters for the non-linear solver
- `print_error` – if `True`, the L^2 discrepancy between the desired state and the numerical solution is printed
- `outputs` – if `True`, output is generated
- `plots` – if `True`, plots of the solutions are generated

```
control.Instationary.incompressible_linear_solve(nullspace_p, *, space_p=None, P=None,
solver_parameters=None, auxiliary_sp={}, v_d=None, f=None, div_v=None, div_zeta=None,
print_error=True, outputs=True, plots=False)
```

Module for the solution of linear incompressible control problems.

Input:

- `nullspace_p` – nullspace of the corresponding forward stationary incompressible problem
- `space_p` – pressure space, if not passed to the constructor
- `P` – preconditioner to apply within the Krylov method (if `None`, default option is employed)
- `solver_parameters` – parameters to pass to the Krylov solver
- `auxiliary_sp` – auxiliary parameters for setting solvers of inner blocks
- `v_d` – when solving non-linear problems, `v_d` is the non-linear residual (adjoint equation)
- `f` – when solving non-linear problems, `f` is the non-linear residual (state equation)
- `div_v` – when solving non-linear problems, `div_v` is the non-linear residual (incompressibility constraint on the state variable)
- `div_zeta` – when solving non-linear problems, `div_zeta` is the non-linear residual (incompressibility constraint on the adjoint variable)
- `print_error` – if `True`, the L^2 discrepancy between the desired state and the numerical solution is printed
- `outputs` – if `True`, output is generated
- `plots` – if `True`, plots of the solutions are generated

```
control.Instationary.incompressible_non_linear_solve(nullspace_p, *, space_p=None,
P=None, solver_parameters=None, auxiliary_sp={}, nl_sp=None, print_error=True,
outputs=True, plots=False)
```

Module for the solution of non-linear incompressible control problems.

Input:

- `nullspace_p` – nullspace of the corresponding forward stationary incompressible problem
- `space_p` – pressure space, if not passed to the constructor
- `P` – preconditioner to apply within the Krylov method (if `None`, default option is employed)
- `solver_parameters` – parameters to pass to the Krylov solver
- `auxiliary_sp` – auxiliary parameters for setting solvers of inner blocks
- `nl_sp` – parameters for the non-linear solver
- `print_error` – if `True`, the L^2 discrepancy between the desired state and the numerical solution is printed
- `outputs` – if `True`, output is generated
- `plots` – if `True`, plots of the solutions are generated

THE PRECONDITIONER MODULE

5.1 Submodules

5.1.1 The `preconditioner.Nullspace` Class

This class defines the nullspace of an operator.

```
preconditioner.Nullspace(abc.ABC)
```

Definition of the nullspace of an operator.

5.1.2 The `preconditioner.MultiBlockSystem` Class

This class constructs and solves a generalized saddle-point problem.

```
preconditioner.MultiBlockSystem(space_0, space_1, block_00, block_01, block_10, block_11,  
*, n_blocks_00=1, n_blocks_11=1, sub_n_blocks_00_0=None, sub_n_blocks_11_0=None,  
nullspace_0=None, nullspace_1=None, form_compiler_parameters=None, CN=False)
```

Class for the solution of a generalized saddle-point system.

Input:

- `space_0` – space to which the state solution belongs
- `space_1` – space to which the adjoint solution belongs
- `block_00` – dictionary representing the (1,1)-block of the generalized saddle-point system
- `block_01` – dictionary representing the (1,2)-block of the generalized saddle-point system
- `block_10` – dictionary representing the (2,1)-block of the generalized saddle-point system
- `block_11` – dictionary representing the (2,2)-block of the generalized saddle-point system
- `n_blocks_00` – number of sub-blocks composing the (1,1)-block
- `n_blocks_11` – number of sub-blocks composing the (2,2)-block
- `sub_n_blocks_00_0` – for time-dependent problems when applying a trapezoidal rule in time, defines the number of sub-blocks of the (1,1)-block for applying the linear transformation employed in the derivation of the preconditioner
- `sub_n_blocks_11_0` – for time-dependent problems when applying a trapezoidal rule in time, defines the number of sub-blocks of the (2,2)-block for applying the linear transformation employed in the derivation of the preconditioner
- `nullspace_0` – nullspace of the first component of the solution

- `nullspace_1` – nullspace of the second component of the solution
- `form_compiler_parameters` – dictionary to be passed to the form compiler
- `CN` – for instationary problems, set `True` if employing a trapezoidal rule discretization in time

```
preconditioner.MultiBlockSystem.solve(u_0, u_1, b_0, b_1, *, solver_parameters=None,  
pc_fn=None)
```

Definition of the linear solver.

Input:

- `u_0` – first component of the solution
- `u_1` – second component of the solution
- `b_0` – first component of the right-hand side
- `b_1` – second component of the right-hand side
- `solver_parameters` – dictionary, contains the parameters for the linear solver
- `pc_fn` – function defining the preconditioner

```
preconditioner.MultiBlockSystem.MultiBlockSystemMatrix(n_blocks_00, n_blocks_11,  
sub_n_blocks_00_0, sub_n_blocks_00_1, sub_n_blocks_11_0, sub_n_blocks_11_1, space_0,  
space_1, spaces, matrices_00, matrices_01, matrices_10, matrices_11, nullspaces, CN)
```

Definition of the generalized saddle-point system.

Input:

- `n_blocks_00` – number of sub-blocks of the (1,1)-block
- `n_blocks_11` – number of sub-blocks of the (2,2)-block
- `sub_n_blocks_00_0` – for time-dependent problems when applying a trapezoidal rule in time, defines the number of sub-blocks of the (1,1)-block for applying the first linear transformation employed in the derivation of the preconditioner
- `sub_n_blocks_00_1` – for time-dependent problems when applying a trapezoidal rule in time, defines the number of sub-blocks of the (1,1)-block for applying the second linear transformation employed in the derivation of the preconditioner
- `sub_n_blocks_11_0` – for time-dependent problems when applying a trapezoidal rule in time, defines the number of sub-blocks of the (2,2)-block for applying the first linear transformation employed in the derivation of the preconditioner
- `sub_n_blocks_11_1` – for time-dependent problems when applying a trapezoidal rule in time, defines the number of sub-blocks of the (2,2)-block for applying the second linear transformation employed in the derivation of the preconditioner
- `space_0` – space to which the first component of the solution belongs
- `space_1` – space to which the second component of the solution belongs
- `spaces` – space to which the full solution belongs
- `matrices_00` – dictionary representing the (1,1)-block of the generalized saddle-point system
- `matrices_01` – dictionary representing the (1,2)-block of the generalized saddle-point system
- `matrices_10` – dictionary representing the (2,1)-block of the generalized saddle-point system

- `matrices_11` – dictionary representing the (2,2)-block of the generalized saddle-point system
- `nullspaces` – nullspace of the full solution
- `CN` – for instationary problems, set `True` if employing a trapezoidal rule discretization in time

`preconditioner.MultiBlockSystem.mult(A, x, y)`

Definition of the matrix–vector multiplication.

Input:

- `A` – matrix representing the system
- `x` – vector of the solution
- `y` – right-hand side

`preconditioner.MultiBlockSystem.Preconditioner(n_blocks_00, n_blocks_11, space_0, space_1, spaces, pc_fn, nullspaces)`

Definition of the preconditioner.

Input:

- `n_blocks_00` – number of sub-blocks of the (1,1)-block
- `n_blocks_11` – number of sub-blocks of the (2,2)-block
- `space_0` – space to which the first component of the solution belongs
- `space_1` – space to which the second component of the solution belongs
- `spaces` – space to which the full solution belongs
- `pc_fn` – function defining the preconditioner
- `nullspaces` – nullspace of the full solution

`preconditioner.MultiBlockSystem.Preconditioner.apply(pc, x, y)`

Definition of the application of the preconditioner.

Input:

- `pc` – preconditioner
- `x` – vector of the solution
- `y` – right-hand side

BIBLIOGRAPHY

- [1] J. Brown, M. G. Knepley, D. A. May, L. C. McInnes, and B. Smith. Composable linear solvers for multiphysics. In *2012 11th International Symposium on Parallel and Distributed Computing*, pages 55–62, 2012.
- [2] S. Leveque and J. W. Pearson. Fast iterative solver for the optimal control of time-dependent PDEs with Crank–Nicolson discretization in time. *Numer. Linear Algebra Appl.*, 29(e2419), 2022.
- [3] S. Leveque and J. W. Pearson. Parameter-robust preconditioning for Oseen iteration applied to stationary and instationary Navier–Stokes control. *SIAM J. Sci. Comput.*, 44:B694–B722, 2022.
- [4] D. Silvester and A. Wathen. Fast iterative solution of stabilised Stokes systems. Part II: Using general block preconditioners. *SIAM J. Numer. Anal.*, 31:1352–1367, 1994.
- [5] W. Zulehner. Nonstandard norms and robust estimates for saddle point problems. *SIAM J. Matrix Anal. Appl.*, 32:536–560, 2011.